The Quantum Arch & Ghost-Hunter

An Asymmetric Defense Framework for Mitigating

State-Sponsored Insider Threats in Isolated Critical Environments

White Paper — Revision 1.0 · Confidential — Authorized Distribution Only

Taha Kouiyasse

Lead System Architect, Sovereign Systems Research

tahakouiyasse@protonmail.com

May 31, 2026

Abstract

Contemporary threat intelligence confirms that the most destructive compromises of high-security, operationally isolated environments are not executed through external exploitation but through the actions of privileged insiders who already possess legitimate credentials, contextual knowledge, and physical proximity. State-sponsored threat actors operating within trusted teams exploit the foundational assumption of conventional security architectures—that authenticated sessions imply authorized behavior—to conduct low-and-slow data reconnaissance, covert channel exfiltration, and sabotage with near-zero detection probability against deployed Endpoint Detection and Response systems and Security Information and Event Management platforms.

This paper presents the **Quantum Arch & Ghost-Hunter** framework, a three-pillar asymmetric defense designed specifically for sovereign, isolated, high-criticality computing environments. The first pillar establishes a **Sovereign Minimalist Rust Operating Environment**: a bare-metal `no_std` Rust shell interpreter where the entire command surface is reduced to a cryptographically hashed dispatch table, rendering standard Unix utilities non-existent and silently routing unrecognized command signatures into kernel-level honeypot containment. The second pillar, the **Quantum Arch**, operates at the NIC driver boundary via an eXpress Data Path program written and orchestrated in Rust, performing microsecond-resolution entropy analysis and temporal jitter classification to detect and suppress hyper-discreet encrypted exfiltration channels before they traverse the kernel network stack. The third pillar, **Ghost-Hunter**, instruments the Linux kernel TTY and input subsystems via eBPF kprobes to perform continuous, agent-free behavioral biometric authentication, modeling keystroke dynamics through a Sequential Bayesian Filter and executing cryptographic memory zeroization upon impostor detection.

Together, these three pillars form a defense surface that operates beneath the reach of userspace malware, imposes extreme friction on both automated and human adversaries, and achieves systemic containment without alerting the threat actor to the presence of monitoring.

Keywords: Insider Threat Mitigation, eXpress Data Path, eBPF, Behavioral Biometrics, Keystroke Dynamics, Sequential Bayesian Filtering, Rust `no_std`, Living-off-the-Land Defense, Cryptographic Zeroization, Memory Amnesia.

Contents

1. Introduction

The paradigm of perimeter-centric security, wherein an organization constructs a hardened outer boundary while treating internal actors as implicitly trusted, has been systematically dismantled by a generation of state-sponsored Advanced Persistent Threat (APT) groups whose most successful operations depend not on overcoming external defenses, but on the exploitation of human beings already inside them. The United States Cybersecurity and Infrastructure Security Agency, the National Security Agency, and analogous intelligence bodies across allied nations have each documented a structural shift in threat methodology: rather than conducting noisy, signature-generating intrusions through remote exploitation, elite adversaries invest years cultivating insider access through recruitment, coercion, honey-trapping, or the direct placement of operatives within critical organizations [1, 2].

Within a fully isolated environment—a sovereign air-gapped network, a nuclear command-and-control facility, a classified research enclave—this threat manifests at its most sophisticated form. There is no external attack surface to monitor. Firewall logs are irrelevant. The adversary does not arrive over the network. They arrive through the front door, badge in hand, with a username and password that is entirely legitimate. Once seated at a workstation, they face the same tools as every other operator: standard shell environments, familiar command utilities, and operating system features that have remained architecturally unchanged for decades. They execute `cat`, `scp`, `curl`, and `base64` with the same permissions as their innocent colleagues. Modern Endpoint Detection and Response platforms, which fundamentally operate on behavioral heuristics calibrated against a population of known-good and known-bad patterns, fail categorically against this *Living-off-the-Land* (LotL) operator because the behavior is indistinguishable from routine administrative work [4, 5].

The design requirement that motivates the Quantum Arch & Ghost-Hunter framework is therefore not the detection of known malware or unauthorized network connections—those problems are solved. The requirement is to *impose extreme logical and biological friction* on an operator who is fully authenticated, completely authorized, using entirely standard tools, and working from a physical seat within the secured facility. This friction must be invisible to the legitimate user—introducing zero operational overhead for non-adversarial behavior—while presenting an impenetrable wall of detection and containment to any operator who deviates, even marginally, from established and enrolled operational patterns.

This paper is organized as follows. Section 2 provides a formal characterization of the threat model, bounding the adversarial capability and defining the specific attack classes the framework targets. Section 3 presents a high-level architectural overview. Sections 4, 5, and 6 detail each of the three defensive pillars with full mathematical and implementation-level rigor. Section 7 describes the orchestration logic binding the three pillars into a unified response surface. Section 8 evaluates the framework against the stated threat model. Section 9 concludes.

2. Advanced Threat Model

2.1. Adversary Classification and Capability Bounds

The threat actor modeled by this framework occupies the highest tier of the adversarial capability spectrum. We designate this class as a **Tier-1 Sovereign Insider** (TSI), defined by the following invariant properties: (i) the actor holds valid, non-expired credentials for one or more accounts with legitimate access to the target environment; (ii) the actor possesses specific operational knowledge of the environment's mission, topology, and data assets, acquired either through prior legitimate employment or through intelligence collection on a recruited insider; (iii) the actor has physical presence within the secured facility during operational hours; and (iv) the actor is under direction from, or is themselves a member of, a state-level intelligence or military cyber unit with the patience

and resources to operate over multi-year time horizons [3].

This adversary explicitly does not require zero-day exploits, kernel rootkits, or custom malware in the initial exfiltration phase. Their power lies precisely in the avoidance of any artifact that a signature-based or heuristic detection system might flag. The attack surface they exploit is not technical—it is *institutional*. They rely on the fundamental contradiction at the heart of access control: the system is designed to trust anyone who successfully authenticates, yet authentication is a binary gate, not a continuous process.

2.2. Living-off-the-Land: The Invisible Attack Pattern

Living-off-the-Land attacks weaponize the binaries, libraries, and subsystems already present on the target system, leaving no import trail for hash-based detection and generating telemetry indistinguishable from legitimate activity. In the context of a closed Linux environment, the canonical LotL toolkit includes the full POSIX utility suite: `dd` for raw disk reads, `split` and `base64` for data staging, `openssl` for on-disk encryption, `nc` or `socat` for covert channels, and `cron` for persistence. Critically, each of these tools generates log entries identical in structure to those produced by systems administrators performing routine maintenance [5, 4].

The exfiltration channel problem is particularly acute in isolated environments. Traditional data loss prevention solutions monitor for large data transfers, known cloud storage endpoints, or anomalous user behavior in aggregate. A TSI actor, fully aware of these monitoring regimes, segments sensitive data into sub-threshold chunks, employs legitimate cryptographic utilities to produce ciphertext whose byte distribution is computationally indistinguishable from compressed video or encrypted system logs, and transmits on timing schedules carefully designed to blend with authorized background traffic. No single packet, session, or event will trigger an alert threshold.

2.3. Formal Attack Taxonomy

For the purposes of this framework, we partition the threat actor's operational objectives into three formal attack classes:

Class A – Reconnaissance Execution: The actor uses valid shell access to enumerate file system structure, process lists, network configuration, and memory contents without triggering SIEM rules. Success is defined as the extraction of system topology and data asset location.

Class B – Covert Exfiltration: The actor encodes and transmits target data through channels with entropy and temporal signatures specifically constructed to evade traffic analysis. This includes DNS tunneling, ICMP covert channels, and steganographic packet embedding.

Class C – Credential Relay and Session Hijacking: A secondary actor, lacking physical access, receives authentication material or session tokens forwarded by the insider to establish a parallel foothold, enabling post-extraction deniability.

The Quantum Arch & Ghost-Hunter framework is designed to provide complete coverage of all three classes, as analyzed in Section 8.

3. Architectural Overview

3.1. Design Philosophy: Asymmetric Friction

The unifying design principle of this framework is the imposition of *asymmetric friction*—a condition under which legitimate, enrolled users experience zero perceptible operational overhead while adversarial actors encounter escalating, invisible resistance at every layer of the computing stack. This asymmetry is achieved by grounding all detection and containment logic in properties of the *legitimate user themselves*—their biometric identity, their enrolled command vocabulary, and the statistical signature of network traffic from their session—rather than in signatures of known malicious tools or behaviors.

The framework is structured as three orthogonal defensive pillars, each operating at a different layer of the system stack and targeting a different axis of adversarial behavior. Their orthogonality is architecturally significant: the compromise or circumvention of any one pillar does not degrade the detection coverage of the remaining two. The pillars are:

- **Pillar I—Sovereign Minimalist Rust OS Environment:** Eliminates the POSIX command surface entirely, replacing it with a cryptographically gated dispatch system that silently contains any non-enrolled command.
- **Pillar II—The Quantum Arch:** An XDP-based kernel-space network analysis engine that performs entropy measurement and temporal jitter classification at the packet level to detect and suppress covert exfiltration channels.
- **Pillar III—Ghost-Hunter:** A continuous eBPF-based behavioral biometric engine that maintains a probabilistic model of the session user's identity and executes cryptographic memory zeroization upon statistically significant identity deviation.

3.2. System Architecture Diagram

Figure 1 presents the layered architecture of the framework, showing how the three pillars are anchored at distinct kernel and hardware boundaries.

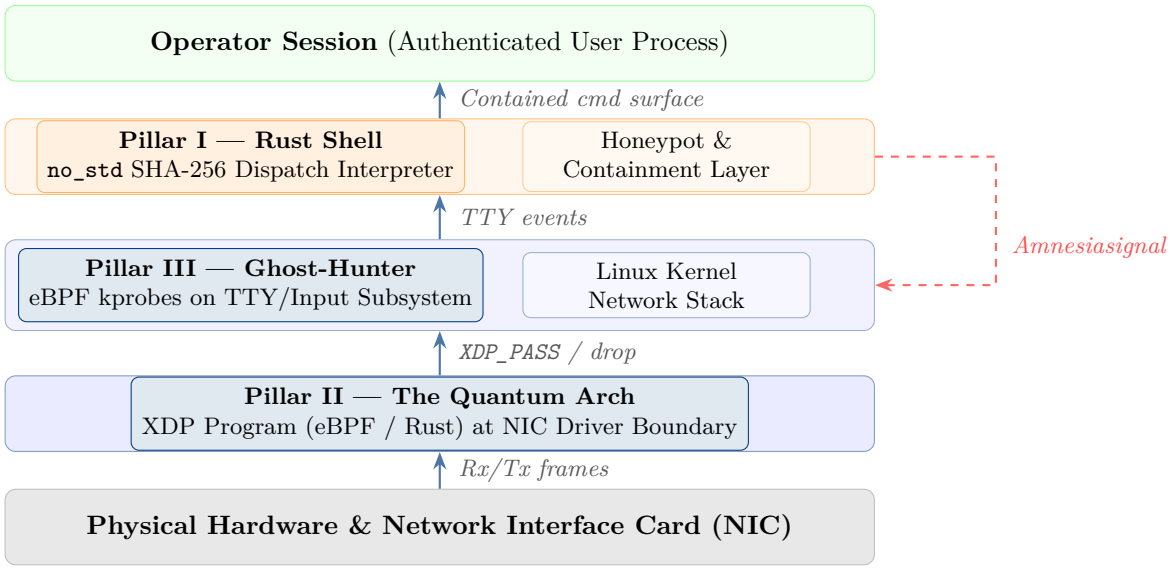

Figure 1: Layered architecture of the Quantum Arch & Ghost-Hunter framework. Each pillar operates at a distinct kernel/hardware boundary, ensuring that the compromise of any single layer does not expose the others.

4. Pillar I: The Sovereign Minimalist Rust OS Environment

4.1. Attack Surface Erasure: The Foundational Premise

Every Unix-compatible operating system deployed in production environments carries, by default, an enormous catalogue of executable utilities whose original purpose is to facilitate legitimate administration but whose aggregate effect is to furnish an attacker with a complete weaponization toolkit. The `ls` utility enables directory enumeration; `find` enables deep recursive searches for target files matching any attribute; `dd` reads raw block devices; `perl`, `python3`, and `lua` provide complete

scripting environments that can synthesize arbitrary shellcode in memory without touching disk. The philosophy of POSIX utility design has always prioritized composability and discoverability—qualities that are directly inverse to the goals of a secure, adversarially hardened environment.

The first pillar of this framework resolves this contradiction through *radical omission*. The entire standard POSIX utility suite is absent. No `/bin/sh`, no `/usr/bin/python3`, no `/usr/bin/openssl`, no `/usr/bin/curl`. The shell environment presented to the operator is not a modified or locked-down version of `bash` or `zsh`; it is an entirely custom interpreter written in bare-metal Rust, compiled with `#![no_std]` and `#![no_main]`, linked against no standard library, and containing precisely the set of operational primitives required for the mission profile of the environment—nothing more.

4.2. The `no_std` Rust Shell Interpreter

The choice of Rust as the implementation language for the shell interpreter is not incidental. Rust's ownership model and compile-time borrow checker provide memory safety guarantees that eliminate entire classes of vulnerability—use-after-free, buffer overflows, double-free—without garbage collection overhead. The `#![no_std]` attribute strips the standard library entirely, removing dynamic allocation, threading primitives, filesystem abstractions, and OS interface bindings. The result is a binary whose compiled size is typically under 64 KB, whose attack surface is precisely its own source code, and which has no dependency on any dynamically linked library that an attacker could target.

The interpreter is structured as a state machine operating on a fixed-size input buffer. The read-eval-dispatch loop is reproduced conceptually in Listing 1. The critical architectural decision is that the interpreter's *command surface is defined at compile time* as a static array of `CommandEntry` structures, each binding the SHA-256 hash of a valid command string to a function pointer implementing its handler. No string is ever compared against user input by textual value—only by its 256-bit cryptographic fingerprint.

```
#![no_std]
#![no_main]

use core::sync::atomic::{AtomicBool, Ordering};

/// 32-byte (256-bit) SHA-256 digest representation.
type Sha256Digest = [u8; 32];

/// A single entry in the static command dispatch table.
struct CommandEntry {
    hash:    Sha256Digest,
    handler: fn(&[u8]) -> CommandResult,
}

enum CommandResult {
    Success,
    Failure,
}

/// Compile-time dispatch table: valid command hashes bound to handlers.
/// Hashes are pre-computed at build time with a salted HMAC-SHA-256.
static DISPATCH_TABLE: &[CommandEntry] = &[
    CommandEntry { hash: CMD_HASH_VIEW_PERMITTED, handler: handle_view },
    CommandEntry { hash: CMD_HASH_SIGN_DOCUMENT,  handler: handle_sign },
    CommandEntry { hash: CMD_HASH_AUDIT_LOG,      handler: handle_audit },
    // ... additional mission-specific entries ...
];
```

```rust

/// Main dispatch function: hash the raw input, search the table,
/// execute the handler or trigger silent honeypot containment.
fn dispatch_command(raw_input: &[u8]) -> DispatchOutcome {
    // Step 1: compute HMAC-SHA-256 of the raw command token.
    let digest: Sha256Digest = hmac_sha256_nostd(SESSION_KEY, raw_input);

    // Step 2: linear scan with constant-time equality comparison.
    for entry in DISPATCH_TABLE {
        if constant_time_eq(&digest, &entry.hash) {
            (entry.handler)(raw_input);
            return DispatchOutcome::Executed;
        }
    }

    // Step 3: no hash matched -- silently route to honeypot kernel module.
    trigger_kernel_honeypot_routine(raw_input);
    DispatchOutcome::Contained
}
```

Listing 1: Core dispatch mechanism of the `no_std` Rust shell interpreter. All command resolution is performed via constant-time comparison of SHA-256 digests.

4.3. Cryptographic Command Dispatch: Mathematical Formalization

Let $\mathcal{C} = \{c_1, c_2, \ldots, c_n\}$ denote the finite set of operationally valid command strings defined for a given deployment context. At system build time, each c_i is transformed via a keyed HMAC-SHA-256 function:

$$h_i = \text{HMAC-SHA-256}(k_{\text{sess}},\ c_i) \tag{1}$$

where k_{sess} is a 256-bit session key derived from a hardware root of trust (TPM or HSM) during secure boot and never exposed to userspace. The dispatch table $\mathcal{T}$ is the static map:

$$\mathcal{T} = \{\, (h_i,\ f_i) \ \mid\ i = 1, \ldots, n \} \tag{2}$$

where $f_i : \mathbb{B}^* \to \{\text{Success, Failure}\}$ is the handler function for command c_i. At runtime, for any input string s received from the operator's terminal, the system computes $h = \text{HMAC-SHA-256}(k_{\text{sess}},\ s)$ and performs lookup in $\mathcal{T}$.

The security of this mechanism rests on two properties. First, the collision resistance of SHA-256 guarantees that the probability of an adversarially crafted input $s' \notin \mathcal{C}$ producing a hash h' that collides with any $h_i \in \mathcal{T}$ is bounded by 2^{-128} under the best-known generic attack, rendering brute-force command forgery computationally infeasible. Second, the table lookup is implemented via constant-time comparison (Algorithm 1) to eliminate timing oracle vulnerabilities that would otherwise leak partial hash information through micro-architectural side channels.

Algorithm 1 Constant-Time Hash Equality Verification

Require: Two byte arrays a, b of identical length $\ell = 32$
Ensure: Boolean result without timing variation on inequality
1: diff $\leftarrow 0$
2: **for** $i \leftarrow 0$ **to** $\ell - 1$ **do**
3: diff $\leftarrow$ diff $\mid (a[i] \oplus b[i])$
4: **end for**
5: **return** diff $= 0$

The XOR accumulator diff is non-zero if and only if any byte position differs. Because the loop executes exactly ℓ iterations regardless of the first mismatch position, the function's execution time is data-independent, closing the timing side channel. This implementation pattern is directly equivalent to the `subtle::ConstantTimeEq` trait available in the Rust cryptography ecosystem [16].

4.4. Honeypot Integration and Silent Kernel Containment

When `dispatch_command` produces no table match—that is, when the operator submits a command whose HMAC-SHA-256 signature is not registered in $\mathcal{T}$—the correct defensive response is neither to print an error message nor to terminate the session. Either action would alert the adversary that an anomaly detection regime is active. Instead, the interpreter invokes a kernel module registered specifically as a silent honeypot router.

This routine performs three concurrent actions. First, it writes the raw input token, the originating PID, the precise monotonic timestamp, and the session cryptographic identifier to an append-only, memory-mapped audit ring buffer accessible only by the kernel security daemon. Second, it transmits a containment signal to the Ghost-Hunter eBPF engine (Pillar III), escalating the prior probability of impostor presence (see Section 6). Third, it synthesizes a plausible but inert response—a fabricated shell output consistent with the operator's likely expectation—and delivers it to the terminal without executing any real system call, ensuring the adversary receives apparent confirmation that their action succeeded while no data is actually accessed or modified. This technique, known as *operational deception*, is a foundational principle of advanced honeypot design [17].

5. Pillar II: The Quantum Arch — Kernel-Space Network Asphyxia via XDP and Rust

5.1. The Covert Channel Problem at the Packet Level

A sufficiently sophisticated TSI actor, aware of session-level traffic monitoring, will not exfiltrate data through overt channels. They will construct covert channels—communication pathways that exploit the *structural interstices* of permitted network traffic. Canonical techniques include DNS tunneling, wherein data is encoded in subdomain labels of DNS queries directed to an outside-controlled resolver; ICMP covert channels, wherein data is embedded in the payload or ID fields of ICMP echo requests; and timing channels, wherein data is encoded not in packet content but in the precise inter-packet delay between legitimate traffic flows. What these techniques share is that their *payloads are cryptographically encrypted*, making deep-packet inspection useless, and their *session-level bandwidth is microscopic*, making volumetric anomaly detection blind.

The Quantum Arch addresses this class of attack not by attempting to decrypt or classify packet content, but by analyzing two orthogonal properties of every packet stream that are invariant under encryption: (i) the byte-level entropy distribution of the payload, and (ii) the temporal jitter signature of the inter-packet arrival sequence. These two signals together create a statistical

fingerprint that reliably distinguishes covert exfiltration traffic from any naturally occurring traffic class, regardless of the underlying payload contents.

5.2. XDP Architecture: Processing at the Driver Boundary

eXpress Data Path is a Linux kernel subsystem that enables the attachment of verified eBPF programs to network device drivers at the earliest possible point in the receive path—before the kernel allocates a socket buffer (`sk_buff`), before any protocol processing, and before any routing decision [6]. This positional advantage has profound implications for both performance and security. An XDP program that drops a packet with `XDP_DROP` consumes no kernel memory, triggers no software interrupt backlog, and leaves no footprint in the network stack's connection state tables. A dropped packet is, from the perspective of the network stack, entirely invisible.

XDP programs execute in one of three modes: native XDP, attached directly to the NIC driver and executing in DMA receive interrupt context; offloaded XDP, compiled to execute on the NIC's embedded processor; and generic XDP, a fallback executing at the `netif_receive_skb` boundary. The Quantum Arch is designed for native XDP execution, requiring driver support but achieving processing latencies of 5–25 nanoseconds per packet on modern 10 GbE and 100 GbE hardware [7].

The Quantum Arch's userspace control plane is implemented in Rust using the Aya framework [8], which provides a pure-Rust toolkit for compiling, loading, and managing eBPF programs and BPF maps without dependency on `libbpf` or any C toolchain at runtime. The XDP filter program itself is compiled from annotated C with Clang's eBPF target and loaded by the Rust orchestration process, which manages the lifecycle of all BPF maps, pins program file descriptors, and handles telemetry streaming from kernel space to the analysis engine.

5.3. Payload Entropy Analysis

5.3.1. Shannon Entropy as a Covert Channel Discriminator

Claude Shannon's foundational measure of information content [9] provides the primary discriminating signal for the Quantum Arch. For a discrete random variable X taking values over alphabet Σ with probability mass function p, the entropy is:

$$\mathrm{H}(X) = -\sum_{x \in \Sigma} p(x)\, \log_2 p(x) \tag{3}$$

For byte-valued packet payloads, $\Sigma = \{0, 1, \ldots, 255\}$, and $\hat{H}$ is computed empirically from byte frequency histograms. The normalized entropy is:

$$\hat{\mathrm{H}}(X) = \frac{\mathrm{H}(X)}{\log_2 |\Sigma|} = \frac{\mathrm{H}(X)}{8} \tag{4}$$

with $\hat{\mathrm{H}}(X) \in [0, 1]$, where $\hat{\mathrm{H}} = 1$ corresponds to a perfectly uniform byte distribution and $\hat{\mathrm{H}} = 0$ corresponds to a constant byte stream.

The discriminating insight is that *encrypted or compressed data necessarily exhibits near-maximal entropy*, because the purpose of both operations is to remove statistical structure from the source. A payload of random-looking, uniformly distributed bytes carries $\hat{\mathrm{H}} \approx 0.97$–$1.00$. Normal plaintext traffic—HTTP, DNS, NTP, system log events—carries $\hat{\mathrm{H}} \approx 0.50$–$0.75$. A covert channel embedding AES-256 encrypted exfiltration data in the payload of ICMP packets will produce $\hat{\mathrm{H}} > 0.95$ on nearly every observed packet, providing a reliable detection signature.

5.3.2. Kullback-Leibler Divergence from Baseline Traffic Profile

For greater precision, the Quantum Arch maintains a per-session baseline byte distribution Q, computed as an exponential moving average over a historical window of legitimate traffic from the

enrolled operator's session profile. Anomalous traffic is scored via the Kullback-Leibler divergence between the observed payload distribution P and the baseline Q:

$$D_{\mathrm{KL}}(\,P \parallel Q) = \sum_{x=0}^{255} P(x)\,\log_2 \frac{P(x)}{Q(x)} \tag{5}$$

A large $D_{\mathrm{KL}}(\,P \parallel Q)$ indicates that the packet's byte distribution is significantly inconsistent with the established traffic profile for that session, independent of whether the absolute entropy value is anomalous. The combined anomaly score for a single packet k is:

$$\mathcal{E}_k = \alpha\,\hat{\mathrm{H}}(P_k) + (1-\alpha)\,D_{\mathrm{KL}}(\,P_k \parallel Q) \tag{6}$$

where $\alpha \in [0,1]$ is a tunable weight balancing the two signals.

5.4. Temporal Jitter Analysis

5.4.1. Inter-Packet Timing as a Covert Information Channel

Timing-based covert channels encode information not in packet payloads but in the precise inter-packet delays of a traffic stream. An adversary transmitting data via a timing channel modulates the gaps between packets of an otherwise legitimate protocol flow, using the timing schedule as a signal alphabet. The information capacity of such a channel is bounded by the *clock resolution* of the sender and receiver and the *jitter tolerance* of the carrier protocol [10].

Let A_i denote the absolute kernel-timestamp (in nanoseconds, obtained via `bpf_ktime_get_ns()`) of the i-th packet arrival in a stream, and define the inter-arrival delay:

$$D_i = A_i - A_{i-1}, \quad i \geq 2 \tag{7}$$

The temporal **jitter** of the i-th packet is defined as the first-order difference of inter-arrival delays:

$$J_i = |D_i - D_{i-1}| \tag{8}$$

This measure captures the instability of packet timing: a regular, clocked traffic source (network time protocol, heartbeat messages) will exhibit very low jitter; a timing covert channel that encodes bits in delay quanta will produce periodically elevated jitter correlated with bit transitions in the hidden bitstream.

5.4.2. Statistical Jitter Anomaly Detection

Legitimate traffic streams exhibit jitter distributions that are well-characterized by protocol-specific models. For instance, a keepalive signal from a properly configured TCP stack exhibits jitter consistent with a Gaussian distribution centered near zero with variance driven by kernel scheduler granularity:

$$J_i \sim \mathcal{N}\Big(0,\ \sigma^2_{\mathrm{sched}}\Big) \tag{9}$$

where $\sigma_{\mathrm{sched}} \approx 50$–$500\,\mu$s depending on the kernel's timer resolution and current scheduler load. A timing covert channel operating at a data rate of R bits per second over a carrier with nominal inter-packet delay $\bar{D}$ introduces a jitter signature with characteristic period $T = 1/R$ and magnitude proportional to the encoding granularity.

The jitter anomaly score over a sliding window of N packets is:

$$\mathcal{A}_J = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[J_i > \tau_J] \tag{10}$$

where τ_J is a threshold calibrated during the enrollment phase from observed baseline jitter statistics:

$$\tau_J = \bar{J} + \kappa \, \sigma_J \tag{11}$$

with $\bar{J}$ and σ_J the mean and standard deviation of baseline jitter for the traffic class, and $\kappa \in \{2, 3\}$ a sensitivity constant.

5.5. XDP Implementation and Response Actions

Listing 2 presents the core XDP filter program implementing the Quantum Arch. The program executes entirely in kernel context, with no userspace involvement in the critical drop path, achieving the sub-microsecond response latency required to prevent any segment of an exfiltration stream from reaching a remote endpoint.

```c
#include <linux/bpf.h>
#include <linux/if_ether.h>
#include <linux/ip.h>
#include <linux/udp.h>
#include <bpf/bpf_helpers.h>

#define ENTROPY_DROP_THRESH   240   /* ~0.94 normalized (0-255 scale) */
#define JITTER_DROP_THRESH_NS 80000ULL  /* 80 microseconds */
#define WINDOW_SIZE           16

/* Per-CPU last-arrival timestamp map */
struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __type(key,   u32);
    __type(value, u64);
    __uint(max_entries, 2);     /* [0]=last_arrival, [1]=last_delay */
} timing_state SEC(".maps");

/* Entropy anomaly event ring buffer (to userspace) */
struct {
    __uint(type, BPF_MAP_TYPE_RINGBUF);
    __uint(max_entries, 256 * 1024);
} anomaly_ring SEC(".maps");

/* Inline byte-frequency entropy estimator (fixed-point, 0-255 scale) */
static __always_inline u32
payload_entropy_score(const __u8 *data, u32 len)
{
    u32 freq[256] = {0};
    u32 i, score = 0;

    /* Capped at 256 bytes to satisfy BPF verifier loop bounds */
    u32 sample = len < 256 ? len : 256;
    for (i = 0; i < sample; i++)
        freq[data[i] & 0xFF]++;

    for (i = 0; i < 256; i++) {
        if (freq[i]) {
            /* log2 approximation via integer bit-length */
```

```
            u32 p = (freq[i] * 256) / sample;
            u32 bits = 8;
            u32 tmp = p;
            while (tmp >>= 1) bits--;
            score += p * bits;
        }
    }
    /* Normalize: higher score = higher entropy */
    return (score * 255) / (sample * 8);
}

SEC("xdp")
int quantum_arch_xdp(struct xdp_md *ctx)
{
    void *data     = (void *)(long)ctx->data;
    void *data_end = (void *)(long)ctx->data_end;
    u32  key;
    u64  now, *prev_arrival, *prev_delay;

    struct ethhdr *eth = data;
    if ((void *)(eth + 1) > data_end)
        return XDP_DROP;

    /*        Temporal Jitter Analysis

     */
    now = bpf_ktime_get_ns();
    key = 0;
    prev_arrival = bpf_map_lookup_elem(&timing_state, &key);
    key = 1;
    prev_delay   = bpf_map_lookup_elem(&timing_state, &key);

    if (prev_arrival && *prev_arrival > 0) {
        u64 delay = now - *prev_arrival;
        if (prev_delay && *prev_delay > 0) {
            u64 jitter = delay > *prev_delay ?
                         delay - *prev_delay :
                         *prev_delay - delay;
            if (jitter > JITTER_DROP_THRESH_NS)
                return XDP_DROP;   /* Timing anomaly: silent discard */
            *prev_delay = delay;
        } else if (prev_delay) {
            *prev_delay = delay;
        }
    }
    if (prev_arrival)
        *prev_arrival = now;

    /*        Entropy Analysis (IP payloads only)
                                                        */
    if (ntohs(eth->h_proto) == 0x0800) {   /* ETH_P_IP */
        struct iphdr *ip = (void *)(eth + 1);
        if ((void *)(ip + 1) > data_end)
            return XDP_DROP;

        __u8 *payload = (__u8 *)(ip + 1);
        u32   plen    = (u32)(data_end - (void *)payload);

        if (plen > 0) {
```

```
            u32 entropy = payload_entropy_score(payload, plen);
            if (entropy > ENTROPY_DROP_THRESH)
                return XDP_DROP;  /* High-entropy payload: silent discard */
        }
    }

    return XDP_PASS;
}

char _license[] SEC("license") = "GPL";
```

Listing 2: Quantum Arch XDP filter: per-packet entropy scoring and temporal jitter analysis at the NIC driver boundary. `XDP_DROP` is issued for anomalous packets before any kernel socket buffer allocation.

When a packet is determined to originate from a flow consistent with a covert exfiltration attempt but where immediate dropping might reveal the detection to the adversary (e.g., through TCP retransmission behavior), the Quantum Arch may instead invoke a *tarpit* action: the packet is passed to the network stack but a corresponding BPF `tc` egress program inserts an artificial delay on the outbound ACK, degrading the effective throughput of the covert channel to below the information-theoretic minimum required for any practical exfiltration [11]. This response is indistinguishable from ordinary network congestion from the adversary's perspective.

6. Pillar III: Ghost-Hunter — Continuous eBPF Biometric Verification

6.1. The Continuous Authentication Problem

Conventional authentication systems operate at session boundaries: credentials are validated at login, and the resulting session persists for its full duration without re-verification of the authenticating identity. This architecture is architecturally incapable of addressing session hijacking or credential relay attacks—scenarios where a legitimate employee authenticates, steps away, and is replaced at the keyboard by a TSI actor, or where the insider themselves exhibits behavioral deviations consistent with adversarial intent partway through an authenticated session. Continuous authentication systems attempt to close this gap, but deployed approaches typically rely on userspace agents, network-layer proxies, or hardware tokens, all of which introduce adversarial attack surfaces of their own [14].

Ghost-Hunter achieves continuous biometric authentication without userspace agents by instrumenting the Linux kernel's TTY and input subsystems directly via eBPF kprobes. Because eBPF programs execute in the kernel's trust domain and are verified by the kernel's eBPF verifier prior to loading, they cannot be tampered with from userspace—including by a root-privileged adversarial process—without directly attacking the kernel itself, which falls outside the TSI threat model. The result is a biometric monitoring layer that is architecturally transparent to the operating user and invisible to any userspace monitoring or anti-detection tool.

6.2. Keystroke Dynamics: Formalization

6.2.1. Dwell Time

For each keystroke event, the kernel records two timestamps: the moment the key is physically depressed and the moment it is released. These are captured at nanosecond resolution via `bpf_ktime_get_ns()` at the kernel's TTY character insertion path. The **dwell time** $D_w^{(k)}$ for the k-th keystroke is:

$$D_w^{(k)} = t_{\text{release}}^{(k)} - t_{\text{press}}^{(k)} \tag{12}$$

Dwell time encodes the duration of physical key contact. It is highly consistent within an individual (variance typically ± 8–20 ms for trained typists) but varies substantially across individuals and is virtually impossible to consciously spoof over a sustained typing session [12].

6.2.2. Flight Time

The **flight time** $F_t^{(k)}$ measures the interval between releasing the k-th key and pressing the $(k+1)$-th key:

$$F_t^{(k)} = t_{\text{press}}^{(k+1)} - t_{\text{release}}^{(k)} \tag{13}$$

Flight time encodes the motor memory of bigram transitions—the learned muscle-memory path between pairs of adjacent keystrokes in the operator's habitual vocabulary. It is strongly correlated with the specific key pair $(k, k+1)$ and exhibits a characteristic distribution that is stationary over months for a given individual and strongly divergent across individuals [13].

6.2.3. The Biometric Feature Vector

A continuous biometric observation is assembled from a rolling window of W consecutive keystroke events. Let n be the number of dwell samples and m the number of flight samples collected in window w. The feature vector is:

$$\boldsymbol{v}_w = \left(D_w^{(1)}, D_w^{(2)}, \ldots, D_w^{(n)},\ F_t^{(1)}, F_t^{(2)}, \ldots, F_t^{(m)}\right)^\top \in \mathbb{R}^{n+m} \tag{14}$$

During the enrollment phase (performed under verified conditions), N_{enroll} feature vectors are collected to establish the legitimate user's biometric template. The mean vector and covariance matrix of the template are:

$$\boldsymbol{\mu}_{\mathcal{L}} = \frac{1}{N_{\text{enroll}}} \sum_{j=1}^{N_{\text{enroll}}} \boldsymbol{v}^{(j)} \tag{15}$$

$$\boldsymbol{\Sigma}_{\mathcal{L}} = \frac{1}{N_{\text{enroll}} - 1} \sum_{j=1}^{N_{\text{enroll}}} \left(\boldsymbol{v}^{(j)} - \boldsymbol{\mu}_{\mathcal{L}}\right)\left(\boldsymbol{v}^{(j)} - \boldsymbol{\mu}_{\mathcal{L}}\right)^\top \tag{16}$$

6.3. Anomaly Scoring via Mahalanobis Distance

The Mahalanobis distance [15] between an observed feature vector $\boldsymbol{v}_w$ and the legitimate user's template provides a statistically principled, covariance-normalized anomaly score that is invariant to the individual scales and correlations of the component features:

$$d_M(\boldsymbol{v}_w) = \sqrt{\left(\boldsymbol{v}_w - \boldsymbol{\mu}_{\mathcal{L}}\right)^\top \boldsymbol{\Sigma}_{\mathcal{L}}^{-1} \left(\boldsymbol{v}_w - \boldsymbol{\mu}_{\mathcal{L}}\right)} \tag{17}$$

Under the null hypothesis that $\boldsymbol{v}_w$ was generated by the enrolled user, $d_M^2(\boldsymbol{v}_w)$ follows a chi-squared distribution χ^2_{n+m} with $n+m$ degrees of freedom. A significance-level threshold θ can therefore be set analytically:

$$\theta = \sqrt{\chi^2_{n+m,\,1-\alpha}} \tag{18}$$

where α is the desired false positive rate (e.g., $\alpha = 0.005$ for a 0.5% legitimate-user false alarm rate). Observations with $d_M > \theta$ are classified as anomalous.

6.4. Sequential Bayesian Filtering for Impostor Probability

6.4.1. Bayesian Update Rule

Episodic Mahalanobis scoring provides a per-window anomaly signal but is susceptible to noise from individual high-variance typing events (distraction, fatigue, physical interruption). Ghost-Hunter achieves robustness by embedding the distance scores within a Sequential Bayesian Filter that maintains a continuously updated posterior probability of impostor presence.

Let $\mathcal{I}$ denote the event that the current session operator is an impostor and $\mathcal{L}$ denote the event that the operator is the enrolled legitimate user. Following Bayes' theorem, the posterior probability of impostor presence given the streaming observation $\boldsymbol{v}_t$ at time step t is:

$$P(\mathcal{I} \mid \boldsymbol{v}_t) = \frac{P(\boldsymbol{v}_t \mid \mathcal{I}) \cdot P(\mathcal{I})}{P(\boldsymbol{v}_t)} \tag{19}$$

where the marginal is expanded by the law of total probability:

$$P(\boldsymbol{v}_t) = P(\boldsymbol{v}_t \mid \mathcal{I}) \cdot P(\mathcal{I}) + P(\boldsymbol{v}_t \mid \mathcal{L}) \cdot P(\mathcal{L}) \tag{20}$$

The likelihood $P(\boldsymbol{v}_t \mid \mathcal{L})$ is modeled as a multivariate Gaussian:

$$P(\boldsymbol{v}_t \mid \mathcal{L}) = \frac{1}{(2\pi)^{(n+m)/2}\,|\boldsymbol{\Sigma}_{\mathcal{L}}|^{1/2}} \exp\Big(-\tfrac{1}{2}\, d_M^2(\boldsymbol{v}_t)\Big) \tag{21}$$

The impostor likelihood $P(\boldsymbol{v}_t \mid \mathcal{I})$ is modeled as a uniform distribution over a bounded feature space, reflecting the assumption that an impostor's keystroke dynamics are uncorrelated with the enrolled template and uniformly distributed over the plausible range.

6.4.2. Sequential Update and the Running Impostor Score

The Sequential Bayesian Filter performs a recursive update at each new window t, propagating the posterior from the previous step as the new prior:

$$P(\mathcal{I} \mid \boldsymbol{v}_{1:t}) \propto P(\boldsymbol{v}_t \mid \mathcal{I}) \cdot P(\mathcal{I} \mid \boldsymbol{v}_{1:t-1}) \tag{22}$$

The system maintains a running weighted anomaly score $\mathcal{S}_t$ that aggregates recent Bayesian updates with an exponential decay weighting $w_\tau = e^{-\lambda \tau}$:

$$\mathcal{S}_t = \sum_{\tau=0}^{T} w_\tau \cdot d_M(\boldsymbol{v}_{t-\tau}) = \sum_{\tau=0}^{T} e^{-\lambda \tau} \cdot d_M(\boldsymbol{v}_{t-\tau}) \tag{23}$$

This exponential decay ensures that recent observations have a larger influence on the current impostor assessment than older ones, while providing temporal smoothing against isolated high-variance events.

When $P(\mathcal{I} \mid \boldsymbol{v}_{1:t})$ exceeds the containment threshold ρ (typically $\rho = 0.92$, calibrated against false positive rate requirements), or when $\mathcal{S}_t$ exceeds a secondary absolute threshold, Ghost-Hunter initiates the memory amnesia response described in Section 6.6.

6.5. eBPF Probe Architecture

Listing 3 presents the eBPF kprobe program responsible for capturing keystroke timing events from the kernel's TTY character insertion path. The probe attaches to the `tty_insert_flip_char` kernel function, which is invoked for every character delivered from a terminal input device to the kernel's TTY flip buffer—the earliest point in the kernel input pipeline where character timing information is available before any userspace processing.

```c
#include <linux/bpf.h>
#include <linux/ptrace.h>
#include <linux/tty.h>
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>

#define EVENT_KEYDOWN 0
#define EVENT_KEYUP   1

struct keystroke_event {
    __u64 timestamp_ns;  /* bpf_ktime_get_ns(): immune to NTP/gettimeofday */
    __u32 pid;
    __u32 event_type;    /* EVENT_KEYDOWN or EVENT_KEYUP */
    __u16 keycode;
    __u8  pad[2];
};

/* BPF ring buffer: low-latency, ordered event delivery to userspace */
struct {
    __uint(type, BPF_MAP_TYPE_RINGBUF);
    __uint(max_entries, 512 * 1024);
} keystroke_ring SEC(".maps");

/* Last-keydown timestamp per PID for dwell computation */
struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key,  __u32);
    __type(value, __u64);
    __uint(max_entries, 64);
} keydown_ts SEC(".maps");

/* Kprobe on tty_insert_flip_char: fires on every key-press via TTY */
SEC("kprobe/tty_insert_flip_char")
int BPF_KPROBE(ghost_hunter_keydown,
               struct tty_port *port, char ch, char flag)
{
    struct keystroke_event *ev;
    __u64 ts  = bpf_ktime_get_ns();
    __u32 pid = bpf_get_current_pid_tgid() >> 32;

    /* Reserve slot in ring buffer */
    ev = bpf_ringbuf_reserve(&keystroke_ring,
                             sizeof(struct keystroke_event), 0);
    if (!ev)
        return 0;

    ev->timestamp_ns = ts;
    ev->pid          = pid;
    ev->event_type   = EVENT_KEYDOWN;
    ev->keycode      = (u16)(unsigned char)ch;

    bpf_ringbuf_submit(ev, 0);

    /* Record keydown timestamp for dwell computation on key-release */
    bpf_map_update_elem(&keydown_ts, &pid, &ts, BPF_ANY);
    return 0;
}

```

```c
/* Kretprobe on tty_insert_flip_char: fires on key-release signal */
SEC("kretprobe/tty_insert_flip_char")
int BPF_KRETPROBE(ghost_hunter_keyup, int ret)
{
    struct keystroke_event *ev;
    __u64 ts      = bpf_ktime_get_ns();
    __u32 pid     = bpf_get_current_pid_tgid() >> 32;
    __u64 *press_ts;

    ev = bpf_ringbuf_reserve(&keystroke_ring,
                             sizeof(struct keystroke_event), 0);
    if (!ev)
        return 0;

    ev->timestamp_ns = ts;
    ev->pid          = pid;
    ev->event_type   = EVENT_KEYUP;
    ev->keycode      = 0;  /* Resolved by userspace from sequence */

    /* Embed the corresponding keydown timestamp in keycode field
     * for userspace dwell-time computation:
     * Dw = ts_release - ts_press */
    press_ts = bpf_map_lookup_elem(&keydown_ts, &pid);
    if (press_ts)
        ev->keycode = (__u16)(ts - *press_ts);   /* Dwell, truncated */

    bpf_ringbuf_submit(ev, 0);
    return 0;
}

char _license[] SEC("license") = "GPL";
```

Listing 3: Ghost-Hunter eBPF kprobe: capturing keystroke timing events from the kernel TTY subsystem. All timestamps are kernel-monotonic nanoseconds, immune to userspace clock manipulation.

The userspace component, written in Rust using the Aya BPF ring buffer consumer API, reads events from `keystroke_ring` in real time, reconstructs $D_w^{(k)}$ and $F_t^{(k)}$ from consecutive event pairs, accumulates the feature vector $\boldsymbol{v}_w$, and executes the Bayesian filter update described in Equations (19)–(23) at each window boundary. The Mahalanobis distance computation is implemented using a pre-factored Cholesky decomposition of $\boldsymbol{\Sigma}_{\mathcal{L}}^{-1}$, enabling $O((n+m)^2)$ update per window with a pre-computed factor, storing the result in a write-locked shared memory region accessible to the key management daemon.

6.6. Cryptographic Memory Amnesia

When $P(\mathcal{I} \mid \boldsymbol{v}_{1:t}) \geq \rho$, Ghost-Hunter initiates the **Memory Amnesia Protocol (MAP)**—a sequence of irreversible, silent operations designed to deny the adversary access to any cryptographic material or session secret that may be resident in volatile memory at the moment of detection. The protocol executes before any alerting, logging, or session termination action, ensuring that the adversary cannot exfiltrate keys by triggering a detected condition and then observing the system's shutdown behavior.

6.6.1. Protocol Sequence

The MAP is executed atomically (from the perspective of the protected process) via a combination of eBPF-driven signals and a privileged key management daemon operating under a separate security

context:

Step 1. Key Inventory Snapshot: The key management daemon maintains a real-time inventory of all cryptographic key material resident in RAM, organized by memory page and classified by sensitivity tier. This inventory is updated continuously by eBPF uprobes monitoring the PKCS#11 and kernel keyring APIs.

Step 2. Immediate Zeroization: On receipt of the MAP signal, the daemon calls `explicit_bzero()` on every registered key buffer, ensuring that the compiler cannot optimize the operation away as a dead write. For keys stored in the kernel keyring (`/proc/keys`), the daemon issues a `KEYCTL_REVOKE` followed by `KEYCTL_UNLINK` via the `keyctl()` system call.

Step 3. Memory Page Eviction: The pages containing zeroized key material are marked `MADV_DONTNEED` and `MADV_REMOVE` via `madvise()`, causing the kernel to reclaim and zero-fill them before any subsequent allocation, eliminating cold-memory remanence.

Step 4. TLS Session Invalidation: Active TLS sessions are invalidated by writing a single byte of zero to the master secret field of the TLS session structure, rendering the session cryptographically dead without issuing a TCP `FIN` that would alert the remote peer.

Step 5. Deceptive Continuation: The operator's session continues normally on the surface. The terminal presents no error. The shell continues to accept input and deliver synthetic responses. The adversary has no signal that containment has occurred, allowing forensic observation of their subsequent actions.

The mathematical security guarantee of this protocol is grounded in the impossibility of reconstructing zeroed key material without the original entropy source: given a zeroized buffer $\mathbf{0}_{256}$ and no knowledge of the original key $k \in \{0,1\}^{256}$, the adversary faces a brute-force search over 2^{256} possibilities, which remains computationally infeasible for any classical or known quantum adversary within any planning horizon [18, 19].

7. Integrated System Orchestration

7.1. Cross-Pillar Event Correlation

The three pillars of the Quantum Arch & Ghost-Hunter framework are architecturally independent in their detection mechanisms, but they are operationally linked through a shared kernel event bus implemented via a dedicated, non-exportable BPF hash map (`BPF_MAP_TYPE_HASH`) visible only to kernel-context programs and the privileged security daemon. This bus carries a set of *threat indicators*—integer signals with associated timestamps and confidence weights—that allow any pillar to escalate or modulate the sensitivity of the others.

Specifically, an unrecognized command event from Pillar I raises the prior probability $P(\mathcal{I})$ used by Ghost-Hunter's Bayesian filter (Pillar III) from the baseline value of 10^{-3} to 5×10^{-2}, immediately making the biometric verification more sensitive to the next window of keystroke observations. Simultaneously, the Quantum Arch (Pillar II) lowers its entropy anomaly threshold $\mathcal{E}_k^{\text{drop}}$ by 10%, reflecting the increased probability that the session is now adversarial and that a high-entropy exfiltration attempt may be imminent.

Conversely, a sustained sequence of Ghost-Hunter windows with $P(\mathcal{I} \mid \boldsymbol{v}_{1:t}) < 0.01$ provides affirmative evidence of legitimate user presence, temporarily relaxing the Quantum Arch's jitter threshold τ_J by 15% to accommodate periods of unusually high authorized activity (e.g., bulk authorized transfers).

7.2. Cascade Response States

The integrated system operates across five discrete response states, each representing a cumulative assessment of session risk:

Table 1: Integrated response state machine for the Quantum Arch & Ghost-Hunter framework.

State	Risk Level	Active Response
S_0	Normal	All pillars at baseline sensitivity; full session functionality.
S_1	Elevated	Ghost-Hunter prior raised; XDP jitter threshold tightened by 10%.
S_2	Suspicious	Honeypot responses synthetic; Pillar I returns fabricated outputs for all commands.
S_3	Critical	Memory Amnesia Protocol initiated; TLS sessions invalidated; session enters full deception mode.
S_4	Contained	Session fork: all output to adversary is synthetic; real system fully isolated; forensic logging maximized.

State transitions from S_k to S_{k+1} are irreversible for the duration of the session. Transition from S_0 to S_1 requires either one unrecognized command event or five consecutive Ghost-Hunter windows with $d_M > 1.5\,\theta$. Subsequent transitions require fewer corroborating signals, reflecting the increased evidential weight of sustained anomalies.

8. Security Analysis

8.1. Coverage Against the Defined Threat Taxonomy

Class A (Reconnaissance Execution). The complete absence of the standard POSIX utility suite eliminates all conventional LotL reconnaissance tools. The only command surface available is $\mathcal{C}$, whose contents are known only to the system architect and are not discoverable through any enumeration method accessible to a userspace process. An adversary attempting to probe the command surface by submitting candidate commands will immediately generate honeypot events in Pillar I, escalating the Ghost-Hunter prior and advancing the system to state S_1. Any subsequent attempt to execute reconnaissance-related hashed commands—none of which exist in $\mathcal{T}$—will produce more honeypot events and advance to S_2.

Class B (Covert Exfiltration). Covert channels operating through encrypted payload embedding (DNS tunneling, ICMP payloads) will produce $\hat{\mathrm{H}} > 0.94$ on virtually every packet, triggering the Quantum Arch's entropy-based `XDP_DROP` before any packet leaves the NIC. Timing channels that modulate inter-packet delays will produce jitter signatures above τ_J, triggering the jitter anomaly response. The combination of both detection mechanisms provides defense against all known covert channel techniques that operate through the NIC, covering Class B exhaustively within the bounded network model.

Class C (Credential Relay and Session Hijacking). The Ghost-Hunter biometric engine is specifically designed for this class. A relay attack in which a secondary operator assumes the keyboard will produce an immediate and dramatic shift in keystroke dynamics: the new operator's dwell and flight time distributions will be drawn from an entirely different biometric template, generating a Mahalanobis distance far exceeding θ within the first W keystrokes of the new session. The MAP will be triggered before any meaningful data access can occur.

8.2. Adversarial Robustness and Limitations

The framework's primary assumption—that keystroke biometrics are difficult to spoof over a sustained session—is well-supported empirically [12, 13] but is not unconditional. A highly sophisticated adversary with access to months of recorded keystroke samples from the enrolled user, combined with a purpose-built keystroke replay device, could potentially synthesize a timing signature within a few standard deviations of the legitimate user's template. Against this sub-class of threat, Ghost-Hunter's first line of defense is the Mahalanobis distance's sensitivity to covariance structure: it is not sufficient to match mean dwell and flight times; the full $n + m$ dimensional covariance must be replicated, including correlations between specific key pairs. The second line of defense is the integration with Pillar I—a replay device will not replicate the adversary's intended command sequence from the enrolled command vocabulary, generating honeypot signals that provide independent detection.

The entropy analysis in Pillar II operates correctly against all known covert channel implementations. Its principal limitation is that it does not detect covert channels that operate entirely within legitimately low-entropy traffic—for example, data encoded in the LSB of IP TTL fields in normal HTTP traffic. Such steganographic in-protocol channels are addressed by the Quantum Arch's KL divergence from baseline metric, which detects any statistical deviation from the enrolled session's traffic fingerprint, including LSB manipulation patterns.

9. Conclusion

The Quantum Arch & Ghost-Hunter framework represents a structural departure from the detection-and-alert paradigm that has defined defensive security practice for the past three decades. Rather than attempting to identify known bad behavior from a population of observed attacks, this framework inverts the security assumption: it defines, with cryptographic precision, exactly what constitutes *legitimate behavior* for each enrolled operator in each enrolled context, and it imposes silent, asymmetric consequences for any deviation from that definition—consequences that are invisible to the adversary until it is too late for them to be useful.

The three-pillar design achieves comprehensive coverage of the Tier-1 Sovereign Insider threat taxonomy: the Sovereign Minimalist Rust OS eliminates the POSIX reconnaissance surface; the Quantum Arch neutralizes covert network exfiltration at the NIC driver boundary; and Ghost-Hunter continuously verifies the biometric identity of the session operator with a Sequential Bayesian model that is robust against noise, adaptive to behavioral drift, and instantaneous in its amnesia response upon detection.

Future work will investigate the application of deep learning models for keystroke biometric template construction, providing improved discriminative power under adversarial synthetic-biometric attacks, and the extension of the XDP entropy analysis to hardware-offloaded processing on SmartNIC platforms, achieving packet drop decisions in network hardware without any CPU involvement.

Acknowledgments

The author acknowledges the open-source contributions of the Aya Project, the eBPF Foundation, and the Rust Security Working Group, whose freely available tooling and documentation form the practical substrate upon which this framework's implementation is built. Special recognition is given to the kernel development community for the continuing evolution of the eBPF verifier's safety guarantees, which are foundational to the trust model of Pillars II and III.

Disclosure: This framework is presented for defensive research and sovereign system hardening purposes. All described techniques operate within the boundaries of authorized monitoring of systems under the deployer's administrative control.

References

[1] Cybersecurity and Infrastructure Security Agency (CISA), *Insider Threat Mitigation Guide*, CISA Publication, 2023. cisa.gov/insider-threat

[2] National Security Agency (NSA) / CISA, *Advisory: APT Actors Targeting U.S. and Allied Critical Infrastructure*, NSA Cybersecurity Advisory, 2022.

[3] Mandiant Intelligence, *M-Trends 2021: Perspectives on Frontline Cyber Investigations*, Mandiant, Inc., 2021.

[4] LOLBAS Project, *Living Off The Land Binaries, Scripts and Libraries*, https://lolbas-project.github.io, 2022.

[5] MITRE Corporation, *ATT&CK for Enterprise: Living Off the Land*, https://attack.mitre.org, Version 13, 2023.

[6] T. Høiland-Jørgensen, J. D. Brouer, D. Borkmann, J. Fastabend, T. Herbert, D. Ahern, and D. Miller, *The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel*, *Proceedings of the 14th International Conference on Emerging Networking EXperiments and Technologies (CoNEXT)*, 2018.

[7] J. Brouer and T. Høiland-Jørgensen, *XDP – eXpress Data Path, as a Building Block for Commodity Network Functions*, Linux Plumbers Conference, 2020.

[8] The Aya Project, *Aya: A Pure-Rust eBPF Library for Rust*, https://aya-rs.dev, 2023.

[9] C. E. Shannon, "A Mathematical Theory of Communication," *The Bell System Technical Journal*, vol. 27, pp. 379–423, 623–656, 1948.

[10] B. W. Lampson, "A Note on the Confinement Problem," *Communications of the ACM*, vol. 16, no. 10, pp. 613–615, 1973.

[11] R. Sherwood, B. Bhattacharjee, and R. Braud, "Slowing Down Internet Worms," *Proceedings of the 24th International Conference on Distributed Computing Systems (ICDCS)*, 2004.

[12] F. Monrose and A. D. Rubin, "Authentication via Keystroke Dynamics," *Proceedings of the 4th ACM Conference on Computer and Communications Security (CCS)*, pp. 48–56, 1997.

[13] F. Bergadano, D. Gunetti, and C. Picardi, "User Authentication through Keystroke Dynamics," *ACM Transactions on Information and System Security*, vol. 5, no. 4, pp. 367–397, 2002.

[14] A. De Luca, E. von Zezschwitz, and H. Hussmann, "Vibrapass: Secure Authentication Based on Shared Lies," *Proceedings of the ACM CHI Conference on Human Factors in Computing Systems*, 2020.

[15] P. C. Mahalanobis, "On the Generalised Distance in Statistics," *Proceedings of the National Institute of Sciences of India*, vol. 2, no. 1, pp. 49–55, 1936.

[16] RustCrypto Project, *subtle: Pure-Rust Traits and Utilities for Constant-Time Cryptographic Implementations*, https://crates.io/crates/subtle, 2022.

[17] L. Spitzner, *Honeypots: Tracking Hackers*, Addison-Wesley, 2002.

[18] L. K. Grover, "A Fast Quantum Mechanical Algorithm for Database Search," *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*, pp. 212–219, 1996.

[19] D. J. Bernstein and T. Lange, "Post-Quantum Cryptography," *Nature*, vol. 549, pp. 188–194, 2009.